

Functional Programming

N28 Fun Coding Workshop Busan

July 4, 2026

Why Functional Programming

What is wrong here?

```
class ShoppingCart:
    def __init__(self, items):
        self.items = items # shares the reference passed in!

    def add_item(self, item):
        self.items.append(item)

    def get_total(self):
        return sum(i["price"] for i in self.items)

# --- Usage ---
default_items = [{"name": "Welcome Gift", "price": 0}]

cart_a = ShoppingCart(default_items)
cart_b = ShoppingCart(default_items)

cart_a.add_item({"name": "Laptop", "price": 1200})

print(cart_b.get_total()) # 1200 🤖 – cart_b now has a laptop in it!
```

Fix

```
from dataclasses import dataclass, field, replace

@dataclass(frozen=True)
class Cart:
    items: tuple

def create_cart(items):
    return Cart(items=tuple(items))

def add_item(cart, item):
    return replace(cart, items=cart.items + (item,)) # new tuple, not mutated

def get_total(cart):
    return sum(i["price"] for i in cart.items)

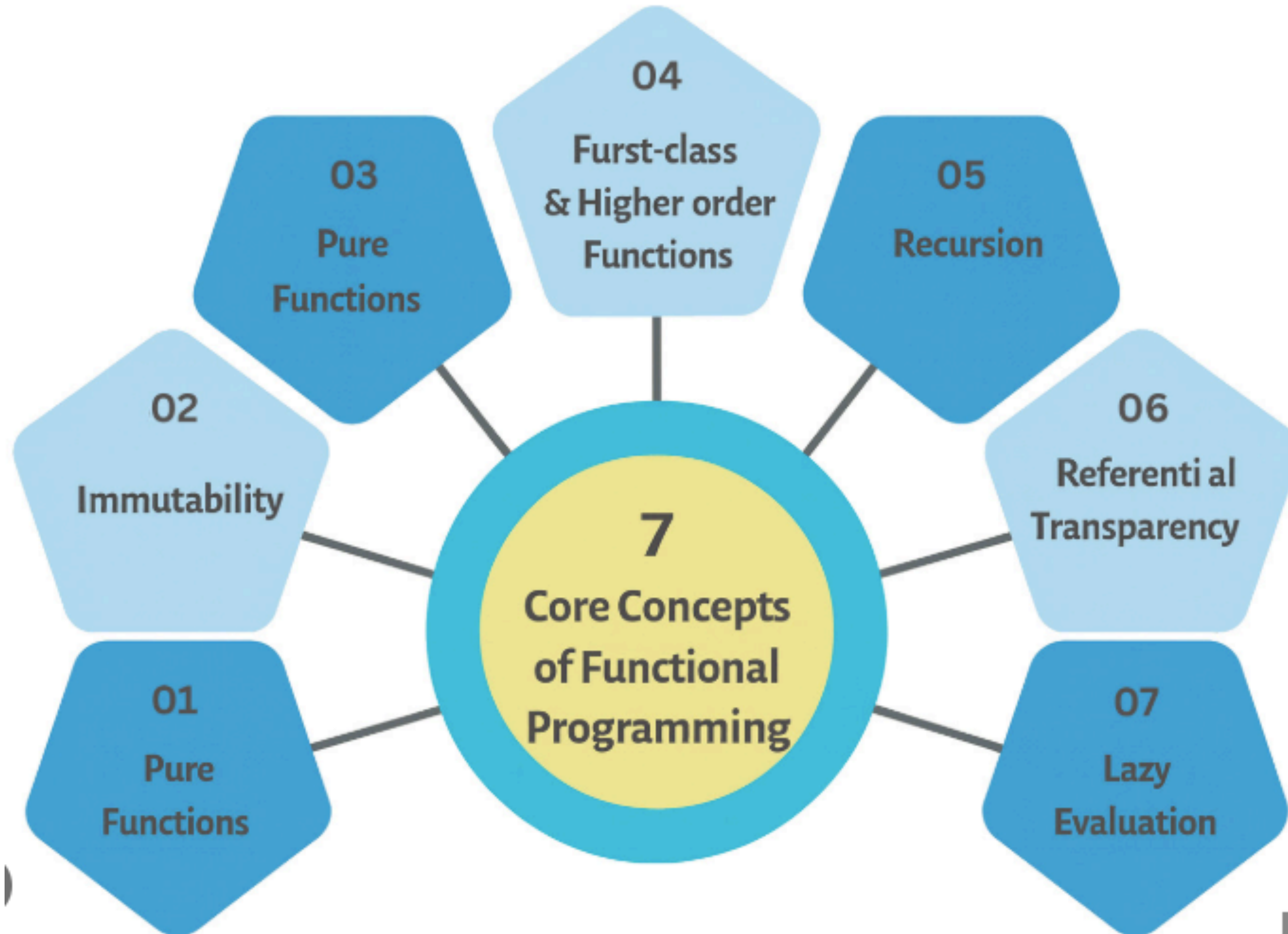
default_items = [{"name": "Welcome Gift", "price": 0}]

cart_a = create_cart(default_items)
cart_b = create_cart(default_items)

updated_cart_a = add_item(cart_a, {"name": "Laptop", "price": 1200})

print(get_total(cart_b))          # 0 ✅ – cart_b untouched
print(get_total(updated_cart_a)) # 1200 ✅
```


Core Concepts



1. Pure Functions

- produce the same output for same arguments
- they have no side-effects (immutability)
- easy to debug and write tests
- better performance because of compiler optimizations

```
def add_item_impure(cart, item):  
    cart.append(item) # mutates the list passed in!  
    return cart
```

```
def add_item_pure(cart, item):  
    return cart + [item] # returns a new list, original untouched
```

2. Immutability

In functional programming, we can't modify a variable after it's been initialized.

We can create new variables – but we can't modify existing variables

```
x = 5
y = x
x = x + 1

print(x)  # 6
print(y)  # 5 – y wasn't affected
```

```
a = [1, 2, 3]
b = a      # b points to the SAME list, not a copy
a.append(4)

print(a)  # [1, 2, 3, 4]
print(b)  # [1, 2, 3, 4] – b changed too!
```

```
s = "hello"
s[0] = "H"  # TypeError: 'str' object does not support item assignment
```

3. Functions are First-Class Citizens

- assign them to variables,
- pass them as arguments,
- return them from other functions,
- store them in data structures.

In OOP, if you want to pass around behavior, you wrap it in an object. In FP, behavior already IS a value — no wrapping required.

```
# OOP way: needs a class hierarchy
class DiscountStrategy:
    def apply(self, price): ...

class NoDiscount(DiscountStrategy):
    def apply(self, price): return price

class TenPercentOff(DiscountStrategy):
    def apply(self, price): return price * 0.9
```

```
# FP way: just pass the function
def no_discount(price): return price
def ten_percent_off(price): return price * 0.9

def checkout(price, discount_fn):
    return discount_fn(price)

print(checkout(100, ten_percent_off)) # 90
```


4. Recursion

Recursion matters in FP because loops depend on mutation

- No loop counters to get wrong - off-by-one errors
- Many core FP data structures (linked lists, trees) are naturally recursive
- Composability — recursive functions are just functions you can pass them around

```
# Imperative: mutation-based loop
def sum_list(numbers):
    total = 0          # mutable variable
    for n in numbers:
        total += n     # mutates on every iteration
    return total
```

```
# Recursive: no mutation at all
def sum_list(numbers):
    if not numbers:
        return 0
    return numbers[0] + sum_list(numbers[1:])
```

A caveat: recursion has a real cost in Python

5. Referential Transparency

Referential transparency means: an expression can be replaced with its computed value, anywhere in the program, without changing what the program does.

```
def square(x):  
    return x * x  
  
result = square(5) + square(5)  
# This is identical to:  
result = 25 + 25  
# Which is identical to:  
result = 50
```

```
counter = 0  
  
def next_id():  
    global counter  
    counter += 1  
    return counter  
  
result = next_id() + next_id()
```

Benefits: caching, reasoning, testing

6. Lazy Evaluation

Lazy evaluation means an expression isn't computed until its value is actually needed

```
# Eager – this list is fully computed right now, all 10 million values  
big_list = [x * 2 for x in range(10_000_000)]  
print(big_list[0]) # even though we only wanted the first item!
```

```
# Lazy – nothing is computed until you ask for it  
big_gen = (x * 2 for x in range(10_000_000)) # generator expression  
print(next(big_gen)) # 0 – only ONE value computed, the rest untouched
```

- **Efficiency** — you never compute values you don't end up using
- **Infinite data structures become possible** — infinite lists/streams are a normal
- **Decouples definition from execution** — you can *build* a big pipeline of transformations without doing any actual work, then trigger it once at the end. store them in data structures.

A caveat: Python's eager by default

**Laziness is easier to reason about in FP
because pure functions + immutable data
guarantee that deferring a computation
never changes its result**

Now Fun Coding Time!